

Next-character prediction

Names become (context, next character) pairs; a learned embedding table and an MLP score all 27 characters.

THE QUESTION

Given the last few characters of a name, how likely is each possible next character, and how do we write new names with that?

THE INTUITION

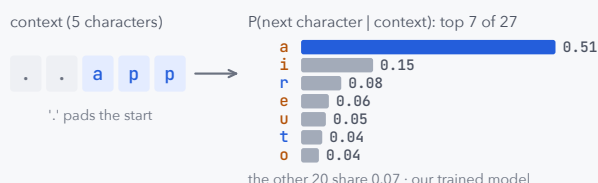
Treat the next character as a 27-class label. Look up a learned vector for each context character, join them, and let an MLP turn them into a softmax. To generate, sample a character, append it and slide the window.

KEY TAKEAWAY

A character model is a softmax classifier on learned features of the context. Train it with cross-entropy; sample from it one character at a time, with temperature setting how bold the samples are.

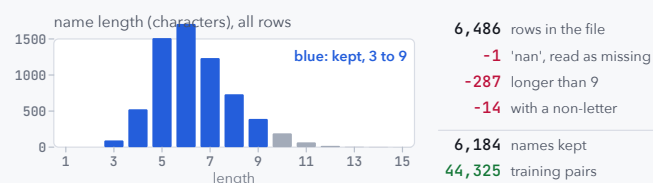
5 context ids → rows of E (27×4) → concatenate: 20 → sin layer: 64 → 27 logits → softmax → loss or sample

1 The next character is one of 27 classes



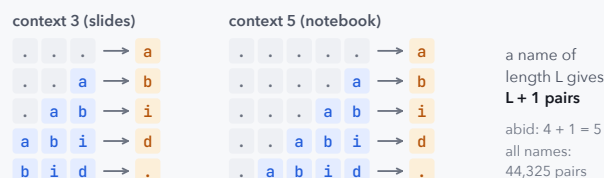
The vocabulary is the 26 lowercase letters plus one token that pads the start and marks the end of a name: . in the notebook (index 0), - in the slides. The model returns a probability for each of the 27, the softmax classifier of the logistic regression sheet with 27 classes. Trained on names, ours follows "app" with a (0.52) rather than the l of "apple".

2 The data: 6,184 Indian first names



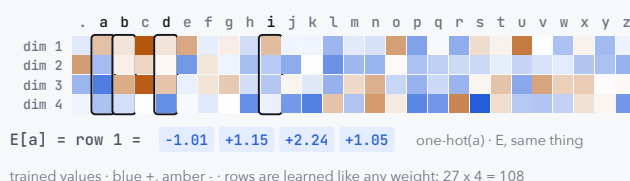
The notebook lowercases the names, removes spaces and keeps words of 3 to 9 letters only (the slides say 4 to 10): 6,184 names, mean length 6.2. We shuffle them with a fixed seed and hold out 10% of the names (619) to measure generalisation; the notebook trains on all of them.

3 A name of length L gives $L + 1$ pairs



Pad the start with ., then slide: each step's context predicts the next character, and the last target is ., so the model also learns where names stop. The block size is the context length: 3 in the slides, 5 in the notebook. All names give X of shape $44,325 \times 5$ (integers) and 44,325 targets y .

4 An embedding table is a learned lookup



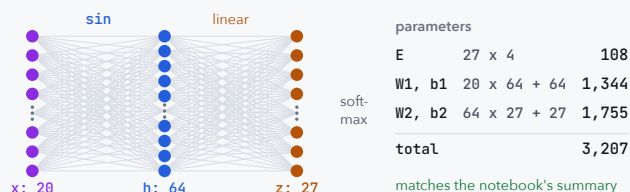
Each character owns a row of $E \in \mathbb{R}^{27 \times K}$; the notebook uses $K = 4$ (the slides draw $K = 2$). Row i equals $e_i^T E$ for the one-hot e_i , so a step changes only the rows that were used. E starts random, $\mathcal{N}(0, 1)$, and backprop trains it with the MLP; the hope is that similar characters end up close, as king — man + woman $\approx$ queen for word vectors.

5 Concatenation keeps the order



$x = [E_{c_1}, \dots, E_{c_5}] \in \mathbb{R}^{20}$, block size times K ; the slides' "abi" with $K = 2$ gives 6 numbers. A sum or mean would forget which slot held which character. The input width of W_1 is block $\times K$, so a new block size means a new network.

6 An MLP turns 20 numbers into 27 scores



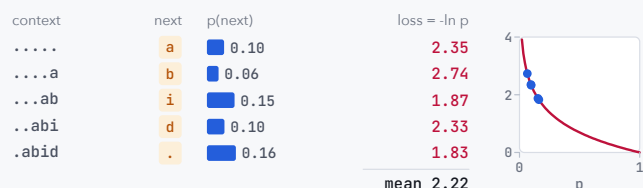
$$h = \sin(xW_1 + b_1), \quad z = hW_2 + b_2, \quad p_k = \frac{e^{z_k}}{\sum_{j=1}^{27} e^{z_j}}$$

The notebook's activation is sin; tanh or ReLU work as well (the multi-layer perceptrons sheet). Of the 3,207 parameters, 108 are the table. Its larger residual model (hidden 128, batch norm, dropout) has 39,815 and reaches a training loss of 1.71.

Fit with cross-entropy, sample one step at a time

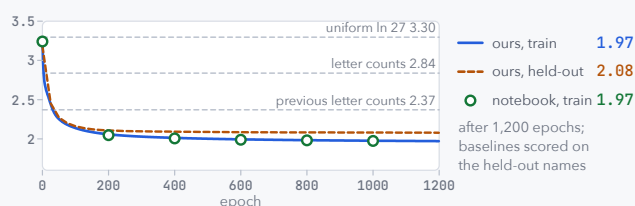
Loss per step, training against counting baselines, learned embeddings, temperature, the sampling tree, new names.

7 Each pair costs $-\ln p$ of its target



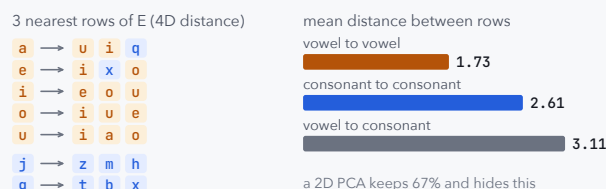
For pair i the loss is $-\ln p_{y_i}$; training minimises the mean over all pairs, the multi-class negative log-likelihood of the logistic regression sheet. A uniform guess costs $\ln 27 = 3.30$ nats. On "abid" the dearest step is b after a ($p = 0.06$).

8 The MLP beats counting letters



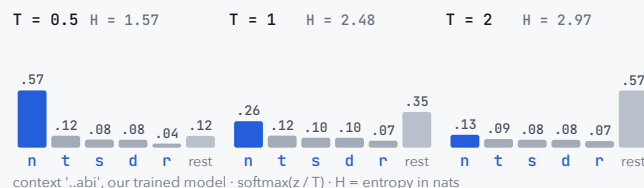
As in the notebook: AdamW (learning rate 0.001, weight decay 0.01), batches of 4,096, 1,200 epochs. Our NumPy run tracks its printed losses, 3.24 to 1.97. On held-out names the MLP scores 2.08, below counts of single letters (2.84) and of the previous letter (2.37). The 0.11 gap to the training loss (the only loss the notebook prints) is memorisation.

9 Trained embeddings group the vowels



Nothing told the model which letters are vowels, yet in the trained E every vowel's nearest row is another vowel, and vowels sit closer to each other than to consonants. Characters that play similar roles in names get similar rows: the representation-learning idea of the slides.

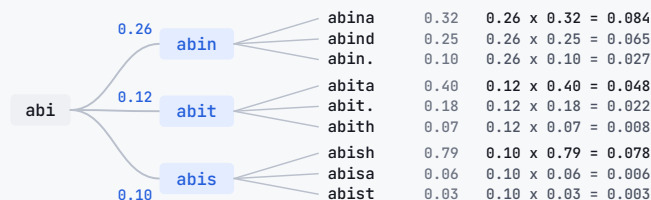
10 Temperature divides the logits



$$p_k(T) = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}} \propto p_k^{1/T}$$

$T = 1$ is the trained model; $T \rightarrow 0$ always picks the top character (greedy); $T \rightarrow \infty$ tends to uniform, entropy $\ln 27 = 3.30$. The ranking never changes, only the gaps.

11 Generation walks down a tree



Start from, sample a character, append it, slide the window, and stop at . (or at the notebook's cap of 10 characters). A name's probability is the product of its steps. Sampling takes one branch per step, so each run gives a new name; greedy decoding always gives the same one.

12 Low T copies names, high T makes noise

	$T = 0.5$	$T = 1$	$T = 2$
copies of training names	24%	6%	1%
distinct among 500	87%	100%	100%
mean length	5.7	6.2	7.1
hit the 10-character cap	1.2%	5.2%	25.6%

new names (not in the data)	shahad aazid	ujmam rojvilal	gidil byomna
-----------------------------	-----------------	-------------------	-----------------

our trained model, 500 samples per temperature, seeded

Quoted from the notebook: before training it wrote "jomngybshs" and "wrhkwunynm"; after, "parvitiya", "garwana" and "hidast". Its "jarmthnaxz" stopped at the 10-character cap, not at ..

13 Pause and think

One token for start and end? It pads and it stops.

Longer context? A wider W_1 : a new net.

One-hot inputs? $e_i^T E$ is the lookup.

Train 1.97, held-out 2.08? Report held-out.

Why concatenate? A mean loses the order.

Greedy or sampled? One name, or many.

names $\rightarrow$ (context, next) pairs · look up and concatenate embeddings · MLP $\rightarrow$ 27 logits · cross-entropy · sample from softmax(z/T), append, slide

Before: the automatic differentiation sheet; next: the convolutional networks sheet. Source: lecture slides "Next token generation" with handwritten notes (after Karpathy's "Neural Networks: Zero to Hero") and the names notebook (nipunbatra.github.io/ml-teaching). Numbers: our seeded NumPy rebuild of its model, 10% of names held out. Corrected: the slides' temperature table (its columns do not follow softmax(z/T)) and the notes' seven pairs from "aabid" (six).